

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
TUGAS 1**



Tugas 1

Oleh:

Aufa Thariq Akram

2510817310014

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
TUGAS 1

Laporan Praktikum Algoritma & Struktur Data Tugas 1 ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Aufa Thariq Akram
NIM : 2510817310014

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Fadhil Syahdama Mahatma Putra
NIM. 2410817210026

Andreyan Rizky Baskara, S.Kom., M.Kom
NIP. 199307032019031011

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
TUGAS 1	1
A. Source Code	1
B. Output Program	13
C. Pembahasan	15
LINK GITHUB	19

DAFTAR GAMBAR

Gambar 1 Screenshot Output Program Tugas 1 Modifikasi 1	13
Gambar 2 Screenshot Output Program Tugas 1 Modifikasi 2	13
Gambar 3 Screenshot Output Program Tugas 1 Modifikasi 3	14
Gambar 4 Screenshot Output Program Tugas 1 Modifikasi 4	14

DAFTAR TABEL

Tabel 1. Source Code Tugas 1	1
------------------------------------	---

TUGAS 1

Berdasarkan kode yang sudah dibuat pada modul 3 lakukan modifikasi pada kode berikut :

1. Modifikasi tambahdepan()

- Dapat menerima beberapa input data dalam satu baris
- Data dipisahkan dengan spasi
- Urutan data pada linked list harus sama dengan urutan input

2. modifikasi hapusbelakang()

- Dapat menghapus node ke-N dari belakang
- Jika N lebih besar dari jumlah node, maka node yang dihapus adalah head

3. modifikasi hapusdepan()

- Dapat menghapus node pada posisi ke-N dari depan
- Jika N lebih besar dari jumlah node, maka node yang dihapus adalah tail

4. modifikasi reset()

- Node dihapus satu per satu
- Setiap proses penghapusan harus menampilkan pada layar node beberapa dan isinya

A. Source Code

Tabel 1. Source Code Tugas 1

1	#include <iostream>
2	#include <stdlib.h>
3	#include <sstream>
4	
5	using namespace std;
6	
7	typedef struct TNode {
8	string data;
9	TNode *next;
10	} TNode;
11	
12	TNode *head, *tail;
13	
14	int pil;

```

15 char pilihan[10];
16 string dataBaru, dataDelete;
17
18 void init();
19 int isEmpty();
20
21 void tambahDepan();
22 void tambahBelakang();
23 void hapusDepan(int n = -1);
24 void hapusBelakang(int n = -1);
25 void tampilkan();
26 void reset();
27 void cariData();
28 void hapusData();
29 void sisipkanSebelum();
30 void sisipkanSetelah();
31
32 int main() {
33     do {
34         cout << "Single Linked List Circular
35 (SLLC)" << endl;
36         cout <<
37 "===== " << endl;
38         cout << "1. Tambah Depan" << endl;
39         cout << "2. Tambah Belakang" << endl;
40         cout << "3. Hapus Depan" << endl;
41         cout << "4. Hapus Belakang" << endl;
42         cout << "5. Tampilkan Data" << endl;
43         cout << "6. Hapus Semua Elemen" << endl;
44         cout << "7. Cari Data" << endl;
45         cout << "8. Hapus Setiap Data Tertentu" <<
46 endl;
47         cout << "9. Sisipkan Node/Data Baru Sebelum
48 Data Tertentu" << endl;
49         cout << "10. Sisipkan Node/Data Baru
50 Setelah Data Tertentu" << endl;
51         cout << "11. Quit" << endl;
52         cout << "Pilihan : ";
53         cin >> pilihan;
54         pil = atoi(pilihan);
55
56         switch(pil) {
57             case 1:
58                 tambahDepan();
59                 break;

```

```

60         case 2:
61             tambahBelakang();
62             break;
63         case 3:
64             hapusDepan();
65             if(!dataDelete.empty()) {
66                 cout << "Data \"" << dataDelete
67 << "\" yang dituju telah berhasil dihapus." <<
68 endl;
69             }
70             break;
71         case 4:
72             hapusBelakang();
73             if(!dataDelete.empty()) {
74                 cout << "Data \"" << dataDelete
75 << "\" yang dituju telah berhasil dihapus." <<
76 endl;
77             }
78             break;
79         case 5:
80             tampilkan();
81             break;
82         case 6:
83             reset();
84             break;
85         case 7:
86             cariData();
87             break;
88         case 8:
89             hapusData();
90             break;
91         case 9:
92             sisipkanSebelum();
93             break;
94         case 10:
95             sisipkanSetelah();
96             break;
97         case 11:
98             cout << "\nTERIMA KASIH" << endl;
99             cout << "Program was made by Aufa
100 Thariq Akram (2510817310014)." << endl;
101             break;
102         default:
103             cout << "Pilihan tidak valid!" <<
104 endl;

```



```

105         }
106
107         if (pil != 11) {
108             cout << "\nPress Enter to continue!" <<
109 endl;
110             cin.ignore(10000, '\n');
111             if (cin.peek() == '\n') {
112                 cin.ignore();
113             }
114             system("clear");
115         }
116
117     } while (pil != 11);
118     return 0;
119 }
120
121 void init() {
122     head = NULL;
123     tail = NULL;
124 }
125
126 int isEmpty() {
127     if(head == NULL) return 1;
128     else return 0;
129 }
130
131 void tambahDepan() {
132     cout << "Masukkan data : ";
133     while(cin.peek() == '\n' || cin.peek() == ' ')
134 cin.ignore();
135     getline(cin, dataBaru);
136     cin.putback('\n');
137
138     stringstream ss(dataBaru);
139     string token;
140     TNode *tempHead = NULL, *tempTail = NULL;
141
142     while(ss >> token) {
143         TNode *baru = new TNode;
144         baru->data = token;
145         baru->next = baru;
146
147         if(tempHead == NULL) {
148             tempHead = baru;
149             tempTail = baru;

```

```

150         } else {
151             tempTail->next = baru;
152             tempTail = baru;
153         }
154     }
155
156     if(tempHead != NULL) {
157         if(isEmpty() == 1) {
158             head = tempHead;
159             tail = tempTail;
160             tail->next = head;
161         } else {
162             tempTail->next = head;
163             head = tempHead;
164             tail->next = head;
165         }
166         cout << "Data \"" << dataBaru << "\"
167 berhasil dimasukkan di bagian depan." << endl;
168     }
169 }
170
171 void tambahBelakang() {
172     cout << "Masukkan data : ";
173     TNode *baru = new TNode;
174     cin >> dataBaru;
175     baru->data = dataBaru;
176     baru->next = baru;
177
178     if(isEmpty() == 1) {
179         head = baru;
180         tail = baru;
181     } else {
182         tail->next = baru;
183         tail = baru;
184         tail->next = head;
185     }
186     cout << "Data \"" << dataBaru << "\" berhasil
187 dimasukkan dibagian belakang." << endl;
188 }
189
190 void hapusDepan(int n) {
191     dataDelete = "";
192     if(isEmpty() == 0) {
193         if(n == -1) {
194

```

```

195         cout << "Masukkan posisi node dari
196 depan (N) : ";
197         cin >> n;
198     }
199
200     int count = 0;
201     TNode *bantu = head;
202     do {
203         count++;
204         bantu = bantu->next;
205     } while(bantu != head);
206
207     if(n > count) {
208         hapusBelakang(1);
209         return;
210     }
211
212     if(n <= 1) {
213         TNode *hapus = head;
214         dataDelete = hapus->data;
215         if(head != tail) {
216             head = head->next;
217             tail->next = head;
218         } else {
219             init();
220         }
221         delete hapus;
222         return;
223     }
224
225     TNode *prev = tail;
226     bantu = head;
227     for(int i = 1; i < n; i++) {
228         prev = bantu;
229         bantu = bantu->next;
230     }
231
232     dataDelete = bantu->data;
233     prev->next = bantu->next;
234     if(bantu == tail) {
235         tail = prev;
236     }
237     delete bantu;
238 } else cout << "Tidak terdapat data pada Linked
239 List." << endl;

```

```

240 }
241
242 void hapusBelakang(int n) {
243     dataDelete = "";
244     if(isEmpty() == 0) {
245         if(n == -1) {
246             cout << "Masukkan posisi node dari
247 belakang (N) : ";
248             cin >> n;
249         }
250
251         int count = 0;
252         TNode *bantu = head;
253         do {
254             count++;
255             bantu = bantu->next;
256         } while(bantu != head);
257
258         if(n > count) {
259             hapusDepan(1);
260             return;
261         }
262
263         int targetPos = count - n + 1;
264         if(targetPos <= 1) {
265             hapusDepan(1);
266             return;
267         }
268
269         TNode *prev = tail;
270         bantu = head;
271         for(int i = 1; i < targetPos; i++) {
272             prev = bantu;
273             bantu = bantu->next;
274         }
275
276         dataDelete = bantu->data;
277         prev->next = bantu->next;
278         if(bantu == tail) {
279             tail = prev;
280         }
281         delete bantu;
282     } else cout << "Tidak terdapat data pada Linked
283 List." << endl;
284 }

```

```

285
286 void tampilkan() {
287     if(isEmpty() == 0) {
288         TNode *bantu = head;
289         do {
290             cout << bantu->data << ' ';
291             bantu = bantu->next;
292         } while(bantu != head);
293         cout << endl;
294     } else cout << "Tidak terdapat data pada Linked
295 List." << endl;
296 }
297
298 void reset() {
299     if(isEmpty() == 0) {
300         TNode *bantu = head, *hapus;
301         int count = 1;
302         do {
303             hapus = bantu;
304             bantu = bantu->next;
305             cout << "Proses penghapusan: Node ke-"
306 << count << " dengan isi data \"" << hapus->data <<
307 "\" telah dihapus." << endl;
308             delete hapus;
309             count++;
310         } while(bantu != head);
311
312         init();
313         cout << "Seluruh elemen pada Linked List
314 telah dibersihkan." << endl;
315     } else cout << "Tidak terdapat data pada Linked
316 List." << endl;
317 }
318
319 void cariData() {
320     if(isEmpty() == 0) {
321         string cari;
322         cout << "Masukan data yang ingin dicari :
323 ";
324         cin >> cari;
325
326         TNode *bantu = head, *bantuTampilkan;
327         bool apaDitemukan = false;
328
329         do {

```

```

330         if(cari == bantu->data) {
331             cout << "Setiap data yang berada di
332 dalam tanda kurung siku ([...]) merupakan data yang
333 anda cari." << endl;
334             cout << "Linked List : ";
335             bantuTampilkan = head;
336
337             do {
338                 if(cari == bantuTampilkan-
339 >data)
340                     cout << "[" <<
341 bantuTampilkan->data << "]" ";
342                     else
343                         cout << bantuTampilkan-
344 >data << ' ';
345                         bantuTampilkan =
346 bantuTampilkan->next;
347                     } while(bantuTampilkan != head);
348
349                     apaDitemukan = true;
350                     cout << endl;
351                     break;
352             }
353             bantu = bantu->next;
354         } while(bantu != head);
355
356         if(!apaDitemukan)
357             cout << "Data \"" << cari << "\" tidak
358 ditemukan pada Linked List." << endl;
359     } else cout << "Tidak terdapat data pada Linked
360 List." << endl;
361 }
362
363 void hapusData() {
364     if(isEmpty() == 0) {
365         string cari;
366         cout << "Masukkan data yang ingin dihapus :
367 ";
368         cin >> cari;
369
370         TNode *bantu = head, *sebelum = tail,
371 *hapusArr[225];
372         int hitung = 0;
373         bool apaDitemukan = false;
374

```

```

375         do {
376             if(cari == bantu->data) {
377                 hapusArr[hitung++] = bantu;
378                 apaDitemukan = true;
379             }
380             bantu = bantu->next;
381         } while(bantu != head);
382
383         if(apaDitemukan) {
384             for(int i = 0; i < hitung; i++) {
385                 if(hapusArr[i] == head) {
386                     hapusDepan(1);
387                 } else if(hapusArr[i] == tail) {
388                     hapusBelakang(1);
389                 } else {
390                     bantu = head;
391                     sebelum = tail;
392                     while(bantu != hapusArr[i]) {
393                         sebelum = bantu;
394                         bantu = bantu->next;
395                     }
396                     sebelum->next = bantu->next;
397                     delete bantu;
398                 }
399             }
400             cout << "Setiap data \"" << cari << "\"
401 yang terdapat pada Linked List telah dihapus." <<
402 endl;
403         } else cout << "Data \"" << cari << "\"
404 tidak ditemukan pada Linked List." << endl;
405         } else cout << "Tidak terdapat data pada Linked
406 List." << endl;
407     }
408
409     void sisipkanSebelum() {
410         if(isEmpty() == 0) {
411             TNode *bantu = head, *sebelum = tail;
412             string nextData;
413             bool apaAda = false;
414
415             cout << "Sisipkan data baru sebelum data :
416 ";
417             cin >> nextData;
418
419             do {

```

```

420         if(nextData == bantu->data) {
421             apaAda = true;
422             break;
423         } else {
424             sebelum = bantu;
425             bantu = bantu->next;
426         }
427     } while(bantu != head);
428
429     if(apaAda) {
430         cout << "Masukkan data yang ingin
431 ditambahkan : ";
432         cin >> dataBaru;
433
434         TNode *baru = new TNode;
435         baru->data = dataBaru;
436         baru->next = bantu;
437
438         sebelum->next = baru;
439         if(bantu == head) {
440             head = baru;
441         }
442         cout << "Data \"" << dataBaru << "\"
443 berhasil disisipkan sebelum data \"" << nextData <<
444 "\"." << endl;
445     } else {
446         cout << "Tidak terdapat data \"" <<
447 nextData << "\" pada Linked List." << endl;
448     }
449     } else cout << "Tidak tedapat data pada Linked
450 List." << endl;
451 }
452
453 void sisipkanSetelah() {
454     if(isEmpty() == 0) {
455         TNode *bantu = head;
456         string prevData;
457         bool apaAda = false;
458
459         cout << "Sisipkan data baru setelah data :
460 ";
461         cin >> prevData;
462
463         do {
464             if(prevData == bantu->data) {

```

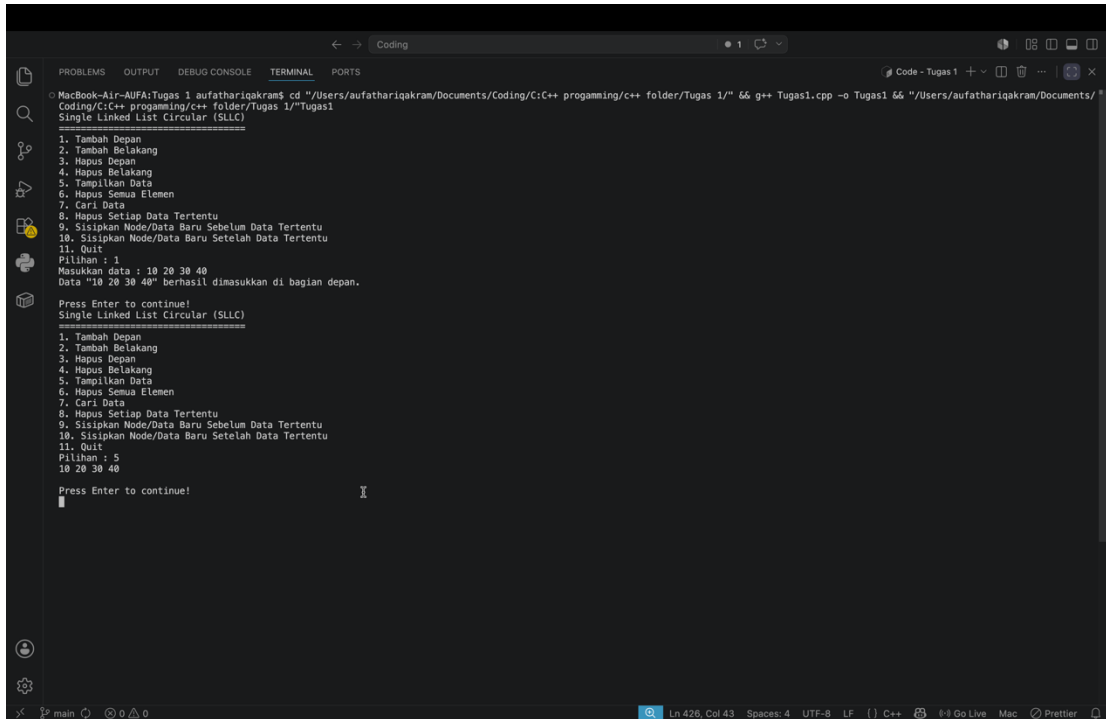


```

465         apaAda = true;
466         break;
467     } else {
468         bantu = bantu->next;
469     }
470 } while(bantu != head);
471
472 if(apaAda) {
473     cout << "Masukkan data yang ingin
474 ditambahkan : ";
475     cin >> dataBaru;
476
477     TNode *baru = new TNode;
478     baru->data = dataBaru;
479     baru->next = bantu->next;
480
481     bantu->next = baru;
482     if(bantu == tail) {
483         tail = baru;
484     }
485     cout << "Data \"" << dataBaru << "\"
486 berhasil disisipkan setelah data \"" << prevData <<
487 "\"." << endl;
488     } else {
489         cout << "Tidak terdapat data \"" <<
490 prevData << "\" pada Linked List." << endl;
491     }
492 } else cout << "Tidak terdapat data pada Linked
493 List." << endl;
494 }

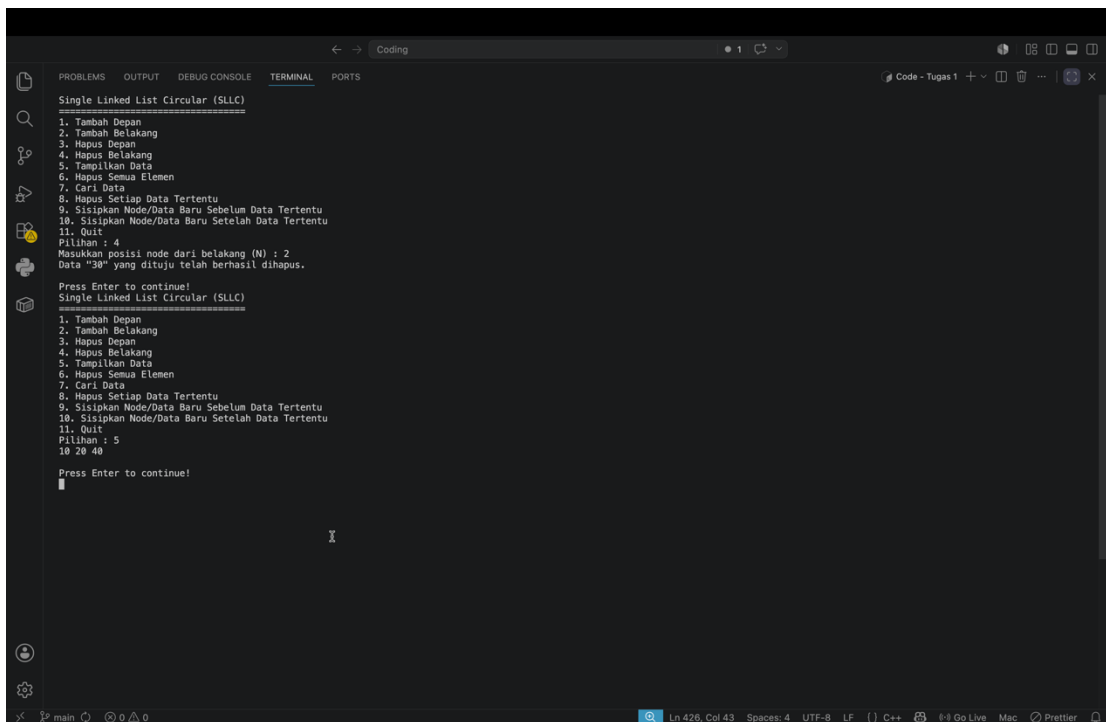
```

B. Output Program



```
MacBook-Air-AUFA:Tugas 1 aufathariqakram$ cd "/Users/aufathariqakram/Documents/Coding/C++ programing/c++ folder/Tugas 1/" && g++ Tugas1.cpp -o Tugas1 && "/Users/aufathariqakram/Documents/Coding/C++ programing/c++ folder/Tugas 1/"Tugas1
Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 1
Masukkan data : 10 20 30 40
Data "10 20 30 40" berhasil dimasukkan di bagian depan.
Press Enter to continue!
Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 5
10 20 30 40
Press Enter to continue!
```

Gambar 1 Screenshot Output Program Tugas 1 Modifikasi 1



```
Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 4
Masukkan posisi node dari belakang (N) : 2
Data "30" yang dituju telah berhasil dihapus.
Press Enter to continue!
Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 5
10 20 40
Press Enter to continue!
```

Gambar 2 Screenshot Output Program Tugas 1 Modifikasi 2

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 3
Masukkan posisi node dari depan (N) : 2
Data "20" yang dituju telah berhasil dihapus.

Press Enter to continue!
Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 5
10 40

Press Enter to continue!
```

Gambar 3 Screenshot Output Program Tugas 1 Modifikasi 3

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Press Enter to continue!
Single Linked List Circular (SLLC)
=====
1. Tambah Depan
2. Tambah Belakang
3. Hapus Depan
4. Hapus Belakang
5. Tampilkan Data
6. Hapus Semua Elemen
7. Cari Data
8. Hapus Setiap Data Tertentu
9. Sisipkan Node/Data Baru Sebelum Data Tertentu
10. Sisipkan Node/Data Baru Setelah Data Tertentu
11. Quit
Pilihan : 6
Proses penghapusan: Node ke-1 dengan isi data "10" telah dihapus.
Proses penghapusan: Node ke-2 dengan isi data "40" telah dihapus.
Seluruh elemen pada Linked List telah dibersihkan.

Press Enter to continue!
```

Gambar 4 Screenshot Output Program Tugas 1 Modifikasi 4

C. Pembahasan

- baris 1 - 5, syntax `#include <iostream>`, `#include <stdlib.h>`, `#include <sstream>`, dan `using namespace std`; berfungsi untuk memanggil pustaka bawaan C++. Pustaka `iostream` untuk menangani input/output standar, `stdlib.h` untuk memanggil perintah sistem (seperti membersihkan layar), dan tambahan pustaka baru yaitu `sstream` (String Stream) yang secara spesifik ditambahkan untuk mendukung Modifikasi 1, di mana pustaka ini memiliki kemampuan untuk memecah kalimat panjang berspasi menjadi kata-kata tunggal yang terpisah.
- baris 7 - 12, syntax `typedef struct TNode` berfungsi untuk merancang struktur data kustom (seperti gerbong) bernama `TNode`. Berbeda dengan Double Linked List, struktur ini merupakan Single Linked List Circular (SLLC) sehingga ia hanya memiliki dua komponen utama: string data untuk menyimpan teks, dan satu buah pointer `*next` yang bertugas menunjuk ke gerbong selanjutnya. Sifat circular (melingkar) berarti gerbong paling belakang (tail) akan selalu menunjuk kembali ke gerbong paling depan (head).
- baris 14 - 29, syntax `TNode *head`, `*tail`;; variabel global, dan deklarasi `void init()`; dan seterusnya berfungsi untuk mendaftarkan variabel penunjuk arah, keranjang data sementara, dan seluruh prototipe fungsi agar dikenali oleh sistem memori komputer sebelum dijabarkan isinya.
- baris 31 - 97, syntax `int main()` berfungsi sebagai blok program utama yang merender antarmuka menu menggunakan perulangan `do while`. Modifikasi kecil terjadi di bagian penahan layar (baris 89-93) menggunakan kombinasi `cin.ignore()` dan `cin.peek()` untuk memastikan program menangkap penekanan tombol Enter dengan sempurna sebelum mengeksekusi `system("clear")` untuk membersihkan layar terminal.
- baris 99 - 107, syntax `void init()` dan `int isEmpty()` berfungsi sebagai pondasi pengaturan. `init` bertugas mereset pointer `head` dan `tail` menjadi hampa (NULL), sedangkan `isEmpty` bertugas mengecek status kehampaan tersebut (mengembalikan nilai 1 jika kosong, dan 0 jika ada isinya).

Pembahasan Bagian yang Dimodifikasi:

- baris 109 - 142, syntax void tambahDepan() (Penerapan Modifikasi 1). Fungsi ini dimodifikasi secara masif agar bisa menerima beberapa input data sekaligus dalam satu baris.
 - Syntax modifikasi: Program menggunakan perintah getline(cin, dataBaru) untuk menangkap seluruh kalimat beserta spasinya. Kalimat ini kemudian dimasukkan ke dalam mesin pemecah kata stringstream ss(dataBaru).
 - Logika modifikasi: Program menggunakan perulangan while(ss >> token) untuk mengekstrak setiap kata. Agar urutan input tidak terbalik saat dimasukkan dari depan, program membuat "rantai sementara" menggunakan pointer tempHead dan tempTail. Setelah semua kata dirangkaiurut di rantai sementara, barulah rantai sementara tersebut disambungkan dan dikaitkan ke head dari Linked List yang sesungguhnya.
- baris 144 - 160, syntax void tambahBelakang() berfungsi untuk menambahkan satu bongkahan node baru di urutan paling belakang dan memastikan pointer next dari node baru tersebut memutar kembali menunjuk ke head.
- baris 162 - 202, syntax void hapusDepan(int n) (Penerapan Modifikasi 3). Fungsi ini dimodifikasi agar dapat menghapus node pada posisi ke-N dari depan.
 - Syntax modifikasi: Parameter int n = -1 ditambahkan. Program pertama-tama menghitung total node (count).
 - Logika modifikasi: Jika angka n yang dimasukkan pengguna lebih besar dari total node yang ada ($n > \text{count}$), maka program akan otomatis melempar tugasnya dengan memanggil fungsi hapusBelakang(1) (menghapus node paling ujung/tail). Jika nvalid (misal $n=2$), program menggunakan perulangan for untuk menjalankan pointer penunjuk tepat ke posisi node ke-2, lalu menghapus node tersebut dan menyambungkan kembali rantai yang terputus.

- baris 204 - 245, syntax void hapusBelakang(int n) (Penerapan Modifikasi 2). Fungsi ini dimodifikasi agar dapat menghapus node ke-N yang dihitung mundur dari belakang.
 - Syntax modifikasi: Sama seperti sebelumnya, program menghitung total node.
 - Logika modifikasi: Jika $n > \text{count}$, maka program akan memanggil hapusDepan(1) (menghapus node paling depan/head). Jika valid, program menggunakan rumus matematika sederhana $\text{targetPos} = \text{count} - n + 1$ untuk mengubah cara pandang "N dari belakang" menjadi "Posisi ke-X dari depan". Setelah titik koordinatnya ketemu, program akan menjalankan pointer penunjuk ke koordinat tersebut dan memutus ikatannya dari memori menggunakan delete.
- baris 247 - 255, syntax void tampilkan() berfungsi untuk mencetak seluruh isi gerbong secara berurutan. Ia berjalan melintasi node menggunakan perulangan do while yang akan terus mencetak bantu->data hingga pointer pemeriksa (bantu) berputar dan kembali bertemu dengan head.
- baris 257 - 271, syntax void reset() (Penerapan Modifikasi 4). Fungsi ini dimodifikasi agar menampilkan proses pembersihan memori secara terperinci (satu per satu).
 - Syntax modifikasi: Penambahan variabel `int count = 1` dan instruksi `cout << "Proses penghapusan: Node ke-" << count << ....`
 - Logika modifikasi: Alih-alih menghapus data secara diam-diam di latar belakang, sistem secara eksplisit menahan pointer yang akan dihapus, mencetak urutan keberapa (count) dan apa isi teksnya (hapus->data) ke layar secara real-time sebelum benar-benar dieksekusi oleh perintah delete hapus;.
- baris 273 - 398, syntax cariData(), hapusData(), sisipkanSebelum(), dan sisipkanSetelah() berfungsi sebagai operasi pencarian dan manipulasi data lanjutan tingkat menengah. cariData menggunakan penanda boolean untuk melacak teks yang sama dan mengapitnya dengan tanda kurung siku []. hapusData memanfaatkan array of pointers (hapusArr) untuk menyimpan

alamat setiap data spesifik yang ingin dimusnahkan secara serentak. Sedangkan fungsi penyisipan bekerja dengan cara memutus rantai dari node target (bantu), dan menyelipkan alamat dari TNode *baru di tengah-tengah rantai yang terputus tersebut.

LINK GITHUB

<https://github.com/IntraGH/Algoritma-dan-Struktur-Data/tree/main/Tugas%201>